

When the Model Screens Off the Harness: Per-Decision Cross-Layer Causal Credit in LLM Agents

[TODO: autores]

August 24, 2026

Abstract

LLM agents interleave decisions from two layers: the *model* (tool calls) and the *harness* (context management, retries, termination). We show, via deterministic counterfactual replay with a *measured* noise floor (exact fidelity under explicit serving premises), that per-decision causal contributions $C(H)$, $C(M)$ and their interaction $I(H, M) = C(H, M) - C(H) - C(M)$ are identifiable on the same trajectory. Interaction is *regime-dependent*: with budget slack, the model action screens off the harness decision ($C_{HM} = C_M$, 0 breaks in 57 points across three configurations); under budget pressure, screening-off breaks (8/65 across three pressure levels; $p = 0.005$ point-level, $p = 0.125$ task-clustered — we report both) at *recurring* decision points, with a non-monotonic dependence on pressure intensity partially mediated by reward saturation. The noise floor replicates on two additional models (Qwen3-1.7B and Qwen3-8B) and on a second environment (a multi-turn MBPP+ adaptation): 4,825 null replays across three models and two environments are all exact (the single fidelity exception occurred in a separate, discarded pre-fix run — 2 inexact nulls of 432, traced to an ASLR memory address leaking into the live context, root-caused, fixed at the source, and re-run: 792/792 exact after the fix). Screening-off itself replicates unconditionally on Qwen3-1.7B (0 raw breaks) and on MBPP+ (66/66 points); on Qwen3-8B it does *not*: raw breaks appear at a similar rate in both regimes (5/28 slack, 6/34 pressure), and the result survives a pre-registered curation of the task pool to the 8B’s competence window (3/27 slack, 3/31 pressure, including a non-saturated slack break) — whether screening-off holds is itself competence-dependent: the shield is a property of the model-harness pair, not of the harness. The re-injection mechanism that fully explains the annotated 4B/8B contrast does not generalize to the curated pool (pre-registered test, reported as failed). This implies that single-layer credit double-counts shielded decisions — not approximately but exactly: at screened points the single-layer harness credit equals the negated interaction, $C(H) = -I(H, M)$, and a coalescence argument — whose occurrence we verify in the recorded traces — shows why determinism makes the equality exact. We then train the harness layer with

interaction-aware credit against outcome-only, single-layer, and untrained controls (dose-matched in total rollouts, 3 seeds, twice — the second run with a pre-registered optimization fix): no credit signal learns the threshold policy that is expressible by the policy class and dominant on held-out tasks (the threshold policy reaches held-out R_{eff} 0.398; every trained arm converges to keep-always at 0.237 or collapses), and credit-driven arms add collapse risk that the outcome-only arm avoids. A pre-registered audit (60 points, two-greedy-replay recomputation, 58/60 sign agreement) rules out the estimator as the cause: the binding constraint is the *training task distribution*, not credit quality — the reward margin between the target policy and the keep-always attractor is statistically zero on the training tasks, and the states where the learned thresholds would fire are unreachable under any context policy. We report this two-stage identified negative result as a design lesson for cross-layer training benchmarks.

1 Introduction

An LLM agent is not a model. It is a model wrapped in a *harness*: a program that manages context, decides when to summarize or truncate history, retries failed calls, and terminates episodes. In deployed agents the harness makes as many decisions per episode as the model does, and those decisions — especially context management — destroy or preserve the information the model conditions on. Yet the dominant training paradigm treats the trajectory as a monolith: outcome-level RL (e.g., GRPO-style policy gradients) assigns one scalar return to every decision, regardless of which layer made it, and recent per-step credit methods attribute within a single layer only. If a harness decision was harmless *because* the model’s next action compensated for it, single-layer credit will bill the harness for an effect the model screened off — a double-counting error that is invisible unless both layers are measured on the same trajectory.

This paper asks the question directly: on the same trajectory, what is the causal contribution of a harness decision, of a model decision, and of their interaction? We answer it with deterministic counterfactual replay. Given a recorded trajectory, we re-execute it from any decision point with exactly one decision changed and the entire prefix forced, and define per-decision credit as $C(d) = R_{\text{orig}} - R_{\text{cf}}$. Replaying pairs and joint flips yields $C(H)$, $C(M)$, $C(H, M)$ and the interaction $I(H, M) = C(H, M) - C(H) - C(M)$ at a single decision pair. Because our replay is *exactly* deterministic (a measured, not assumed, property — and one that fails silently if vLLM’s automatic prefix caching is left on), every nonzero credit is signal.

The headline finding is that the interaction term is *regime-dependent*. When the agent has budget slack, the model action screens off the harness decision exactly: $C_{HM} = C_M$ in 57 out of 57 measured points across three configurations. Under budget pressure the shield cracks — 8 breaks in 65 points, concentrated at recurring decision points — and the dependence on pressure intensity is non-monotonic, partially mediated by reward saturation: interaction is only detectable inside a *competence window* where the agent neither aces nor

fails everything. The practical corollary is negative for naive pipelines: in the slack regime, single-layer harness credit systematically double-counts effects the model absorbs.

We then test whether interaction-aware credit improves *training* of the harness layer, with pre-registered predictions, dose-matched controls, and three seeds per arm — twice. The honest outcome is a two-stage identified negative: optimization instability first, and, after a pre-registered fix, stable training that still fails to discover a threshold policy which is expressible by the policy class and dominant on held-out tasks. A pre-registered audit exonerates the credit estimator; the binding constraint is the training task distribution itself, not attribution. We report this as a design lesson for cross-layer training benchmarks, not as a method contribution.

Contributions.

1. **Deterministic replay infrastructure with a measured noise floor** under explicit identification premises (fixed config, sequential execution, prefix caching off) — including a cautionary finding: vLLM automatic prefix caching silently breaks greedy-decoding determinism across differing serving histories.
2. **Per-decision counterfactual decomposition** of $C(H)$, $C(M)$, and $I(H, M)$ on the same trajectory, and the finding that screening-off is regime-dependent (exact under slack, breaking at recurring points under pressure).
3. **Anatomy of screening-off, with a formal core:** a double-counting identity ($C(H) = -I(H, M)$ exactly at screened points, with $C_{HM} - C_M$ as the correction rule), a coalescence lemma and corollary whose occurrence we *verify in the recorded traces* (72/75 screened points explained; 3 residuals, including 2 reward collisions, declared), and a pivotality argument that unifies the competence window, the mt4 collapse, and the saturation profiles of both auxiliary models; the replications show the shield itself is competence-dependent — the 8B breaks screening where the 4B never does, in the raw data and in a pool pre-registered and curated to the 8B’s competence window. The re-injection mechanism explains the annotated 4B/8B contrast (23/24 and 0/24 points) but fails a pre-registered generalization test in the curated pool, and we scope it accordingly; empirically, a structural component (the sampled a' re-injects destroyed information, 23/24 annotated points) and a closed-loop component (the live downstream harness re-triggers the intervened decision, 22/52 points).
4. **Construct validation by control tasks:** the method separates harmless from harmful harness decisions by construction (11/12), including one audited exception where a harness decision had a *behavioral*, not informational, effect.

5. **A pre-registered training comparison with an identified negative outcome:** interaction-aware credit vs. outcome-only, single-layer credit (a reproduction of the CHILL-Harness principle in our environment), and a zero-credit masking control (de facto an untrained keep-always policy under our greedy tie-break — stated explicitly), dose-matched in total rollouts. Act 1: every trained arm collapses seed-dependently. Act 2 (after a pre-registered optimization fix): stable training, right direction, yet no arm discovers the threshold policy that is expressible and dominant on held-out tasks, and credit-driven arms retain collapse risk (1/3 seeds) that outcome-only avoids (0/3). A pre-registered audit (58/60 sign agreement) rules out the estimator; state-visitation and reward-margin analyses localize the failure in the training task distribution, not in credit quality.

2 Related Work

Counterfactual replay for agent credit is an active area, but every existing system we are aware of attributes within a single layer. CHILL-Harness [3] intervenes on harness decisions, using checkpointed paired replay *offline* to train an amortized effect estimator (replay is disabled at decision time), but never measures the model layer or the interaction; our single-layer training arm (§6) is an explicit reproduction of its principle in our environment, which turns the comparison into a result. CAR [7] formalizes per-step do-operations with Shapley attribution under a replay budget — including interaction *between steps* — but within one decision stream: it never crosses the model/harness boundary. C3 [1] audits credit fidelity against exact replay ground truth (the same validation metric we adopt) and adds an inter-agent influence diagnostic, but decomposes across *agents*, not across the model/harness boundary inside one agent, and its influence measure is a diagnostic, not a training signal. A recent negative result [8] shows that popular step-level credit signals fail to beat chance against replay ground truth; it sets the evidential bar our critic section inherits (dose-matching, pre-registration, heuristic baselines). On the training side, Co-Harness [2], HarnessCompass [9] and HASE [5] jointly optimize harness and weights, but without a per-decision causal signal — they optimize the composition blindly (HarnessCompass’s “cross-component interference” is between components *of the harness*, not causal model–harness interaction). Agent Lightning [6] provides rollout infrastructure for agent RL without addressing attribution. Table 1 summarizes the gap this paper attacks: no prior work measures $C(H)$, $C(M)$ and $I(H, M)$ at the same decision point of the same trajectory against an exact, zero-noise-floor replay — every existing system attributes within a single layer (the harness, the step stream, or the agent), where cross-layer interaction is invisible by construction and single-layer credit silently double-counts effects the other layer absorbs. Where interaction appears at all it is a diagnostic to report, not a quantity whose structure is interrogated or a training signal; and no prior work asks whether the causal structure recovered by attribution is a stable property of the pipeline or of the specific model–harness

Method	counterfactual via replay	measures $C(H)$	measures $C(M)$	$I(H, M)$ on same trajectory	trains with I	structure across models tested
CHILL-Harness	✓ ^{off}	✓	—	—	—	—
CAR	✓	—	✓	— ^s	—	—
C3 v2	✓	(per agent)		—	—	—
Replay audit [8]	✓	—	✓	—	—	~ ^m
Co-Harness / HASE	—	—	—	—	— [*]	—
Ours	✓	✓	✓	✓	tested [†]	✓

Table 1: T1 — positioning. ^{off}Checkpointed paired replay used offline to train an amortized estimator; disabled at decision time. ^sCAR measures step-step interaction (Shapley) *within* one layer, not across the model/harness boundary. ^{*}Joint harness+weights training without per-decision causal measurement. ^mObserves that counterfactual *measurability* is model-dependent, within a single layer; no cross-layer structure. [†]Tested with dose-matched controls and pre-registration; the outcome is an identified negative (§6).

pair. The closest precedent is the observation that counterfactual *measurability* is model-dependent [8]; §7 shows the stronger phenomenon — the structure itself inverts with model competence at identical intervention points.

3 Setup: A Two-Layer Agent with Deterministic Replay

3.1 Agent, decision schema and recorder

The agent is deliberately minimal: a coding agent in a sandboxed environment that reads a task description, writes files, and runs tests. Two layers make decisions. The **model layer** (Qwen3-4B served by vLLM, greedy decoding) chooses tool calls: `write_file`, `run_tests`, `read_file`, `finish`. The **harness layer** is a scripted policy that at every turn makes a `context_policy` decision (`keep_context` vs. `summarize_context`, the latter compressing history to a fixed-length prefix) and a `termination` decision. Every decision — model or harness — is recorded with a typed schema: decision id, layer, state before, available actions, chosen action, observation, token costs, and parent/child links. Reward is deterministic: the fraction of hidden pytest tests passed, so $R \in [0, 1]$ with resolution set by the number of tests. Cost-adjusted reward is $R_{\text{eff}} = R - \lambda \cdot \text{prompt tokens}/10^5$ with $\lambda = 25$, fixed analytically before any training and never recalibrated.

3.2 Replay and interventions

Replay re-executes a recorded trajectory through the *live* agent loop with a forced-action queue: recorded decisions are matched against the loop’s requests

(peek-match on decision type and state hash) and forced, up to the intervention point; from there the loop runs free. An intervention replaces exactly one decision (or, for $C(H, M)$, one harness-model pair). The null intervention — force everything, change nothing — must reproduce R exactly; §4.1 shows it does, and what serving configuration is required for it to. All replays run sequentially against a single vLLM instance with a fixed config.

3.3 Tasks

We use 30 synthetic coding tasks in four designed strata, plus 10 held-out tasks. **V2** and **L** tasks place critical constants beyond the first 240 characters of history, so a summarize decision is *informationally destructive* at known positions; **L** tasks are longer variants. **S** tasks are built so that success requires harness and model decisions to cooperate (synergy by design). **C** tasks are controls: all information needed survives any summary, so the construct predicts $C(H) = 0$. Reward saturation (episodes at $R \in \{0, 1\}$) is endogenous to the number of tests per task; we treat it as a design choice and analyze its consequences explicitly (§4.3, §7).

4 Measuring Cross-Layer Credit

4.1 Noise floor

Every causal claim below is only as good as the replay. We therefore *measure* the noise floor rather than assume it: the pre-registered floor for each configuration is $\max |\Delta R|$ over null replays (replay with no intervention). The floor is defined on the reward R , not on trace equality: in 3 null replays (all in one configuration) the replayed trajectory took one extra retry while reproducing R exactly, and we report this rather than hide it. Across seven dedicated null-replay runs of Qwen3-4B (1,230 nulls), the nulls attached to every interaction run (401 more on Qwen3-4B), two configurations of Qwen3-1.7B (64 nulls), three configurations of Qwen3-8B (810 dedicated + 234 attached), two MBPP+ configurations (1,080 dedicated + 214 attached), and four curated-pool configurations on Qwen3-8B (792 nulls, run after the fix reported below), **every one of 4,825 null replays reproduced R exactly**: the floor is 0.0 in all configurations (Figure 1a). This is a measurement under explicit premises — fixed serving config, strictly sequential requests, prefix caching off — not a guarantee.

A second silent nondeterminism source surfaced later, and we report it the same way: in a discarded pre-fix run of the first hardened pool (excluded from the 4,825: its pool failed the pre-registered curation rule, and its configurations were re-run after the fix), 2 of 432 nulls were inexact — the first fidelity breaks observed in this project (2/5,257 including every null ever run under the stated premises; the APC-contaminated run of Figure 1b violated them and is counted separately). Root cause, identified from the diverging trace: a memory address inside a Python `repr` in the pytest output (ASLR) entered the model’s live con-

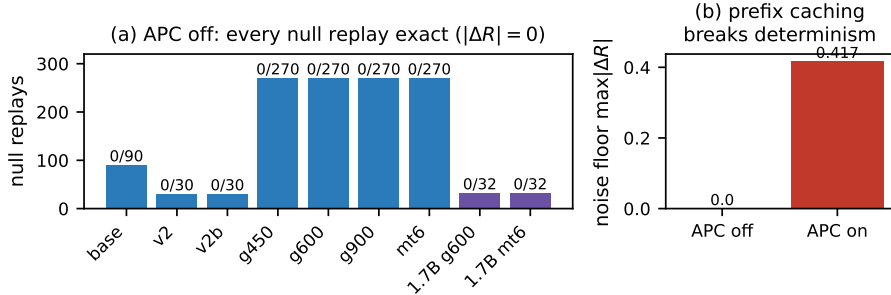


Figure 1: **F1 — Measured noise floor.** (a) Dedicated null-replay runs (Qwen3-4B blue, Qwen3-1.7B purple); together with nulls attached to interaction runs, the Qwen3-8B and MBPP+ replications, and the curated-pool runs, all 4,825 null replays are exact, floor = 0.0. (b) With vLLM automatic prefix caching on, the floor rises to 0.417 (7/270 inexact): greedy determinism breaks silently.

text, and greedy decoding diverged downstream. The sanitizer now normalizes addresses at the source (regression-tested), and the affected configurations were re-run from scratch. Two honesty notes: the 4,033 nulls collected *before* the fix were exact because their pytest outputs happened to contain no address-bearing **reprs** — measured, not guaranteed by sanitization; and “four curated-pool configurations” means 2 budget regimes $\times$ 2 pool halves. The lesson generalizes: replay-grade determinism requires auditing every byte that reaches the context, not just the serving stack. The premises matter. vLLM V1 enables automatic prefix caching (APC) by default, and with APC on, prefill numerics depend on the KV-cache state left by *previous* requests: greedy decoding diverges across identical prompts with different serving histories, even when requests are sequential. In our APC-contaminated run, 7 of 270 null replays were inexact and the floor rose to 0.417 (Figure 1b) — large enough to swamp most true credits. Sequential execution alone is *not* sufficient; APC must be disabled. We flag this because any replay-based attribution system built on default vLLM settings inherits this silent failure.

4.2 $C(H)$ and $C(M)$

With an exact floor, per-decision credits are directly interpretable: any $C \neq 0$ is a real causal effect of that single decision. Figure 2 shows the distributions over 248 harness points and 66 model points, by stratum. Three regularities matter downstream. First, direction is everything: flipping **keep**→**summarize** at an information-critical point yields large positive $C(H)$ (the decision to keep was worth up to +0.89), while the reverse direction is near zero — we therefore never aggregate across directions. Second, credits are sparse: most decisions are causally inert, and mass concentrates at information-destruction points pre-

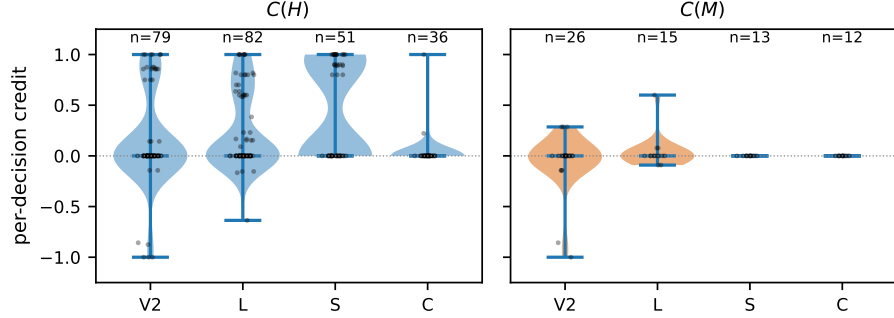


Figure 2: **F2 — Per-decision credit by layer and stratum.** $C(H)$ (248 points) and $C(M)$ (66 points) against an exact zero floor. Strata: V2/L information-destruction, S designed synergy, C controls.

dicted by task design (V2/L strata). Third, $C(M)$ at matched points is frequently nonzero when $C(H)$ is not, foreshadowing the screening-off structure of §4.3.

4.3 Interaction $I(H, M)$ and its anatomy

The central measurement flips a harness decision and a model decision at the same point, alone and jointly. **With budget slack** (max 12 turns; three budget configurations g450/g600/g900), the model action screens off the harness decision *exactly*: $C_{HM} = C_M$ in **57/57** points — the sampled alternative model action re-establishes what the harness destroyed, so the harness flip adds nothing beyond the model flip. **Under budget pressure** (max 4/6/8 turns), screening-off breaks in **8/65** points (slack vs. pressure: $p = 0.005$ point-level hypergeometric; $p = 0.125$ task-clustered sign test over 3 breaking tasks — we report both, and note the point-level test treats points within a task as independent). Breaks recur at the *same* decision points across configurations (`l_log_parser` idx 8 in 3/3 pressure configs; `api_router` idx 1 and `l_vending_machine` idx 11 in 2 each), which is what a real mechanism, rather than noise, predicts.

The dependence on pressure intensity is *non-monotonic*: break rates are 0/21 (mt12), 4/22 (mt8), 3/21 (mt6), 1/22 (mt4) — an inverted U. Our pre-registered monotonicity test failed (Cochran–Armitage $p = 0.38$); we report that failure and analyze the cause. The mediator is **reward saturation**: at mt4 the agent fails most tasks outright (R pinned at 0), so no flip can make things worse and interaction is masked. Conditioning on non-saturated points sharpens the contrast (slack 0/12 vs. pressure 6/19, $p = 0.037$) and partially restores the mt4 rate ($0.045 \rightarrow 0.143$), though the ordering $\text{mt8} > \text{mt6} > \text{mt4}$ persists — saturation mediates the window partially, not fully (Figure 3). At the task-cluster level the conditioned contrast, like the raw one, is weak (paired sign-flip permutation over 4 tasks $p = 0.25$; cluster-label permutation $p = 0.18$);

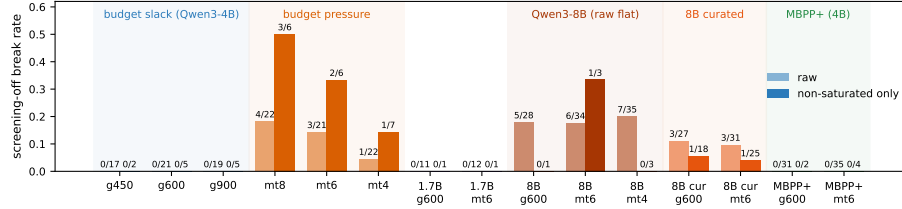


Figure 3: **F3 — Screening-off is regime-dependent, and the regime contrast is model-dependent.** Break rate of $C_{HM} = C_M$ per configuration, raw and conditioned on non-saturated points. Slack: 0/57 raw (0/12 non-saturated). Pressure: 8/65 raw (6/19 non-saturated, $p = 0.037$). Qwen3-1.7B (purple) replicates slack; its pressure null is explained by saturation (only 1 non-saturated point). Qwen3-8B (red) breaks raw screening at a *flat* rate across regimes (5/28, 6/34, 7/35), and the flatness survives the pre-registered curated pool (orange: 3/27 slack, 3/31 pressure) — the competence dependence of §7. MBPP+ (green, Qwen3-4B) replicates screening unconditionally (0/66).

the point-level and mechanism-level evidence carry the claim (§7). We call the resulting picture a *competence window*: cross-layer interaction is detectable only where the agent neither saturates success nor failure.

Anatomy of the shield. The sets in play: across the three slack configurations and the first pressure configuration we measured 78 interaction points (57 slack + 21 at mt6); **75/78 show exact screening-off** ($C_{HM} = C_M$), which under the pre-registered sign test against a 50/50 null gives $P = 2.6 \times 10^{-19}$ — the 3 exceptions are the mt6 pressure breaks. Among the screened points, 52 are non-trivially *shielded*: the harness flip alone has a real effect ($C(H) \neq 0$) that the joint flip erases — these are exactly the double-counting cases. Of the 52 shielded points, 24 carry constant-dictionary annotations that make the re-injection check possible (the remaining 28 are unannotated V2 points, which we declare rather than extrapolate). Among the annotated: **23/24** show *structural re-injection* — the sampled alternative action a' carries the destroyed information across the cut, a property of the estimand (since a' is sampled from the model conditioned on the un-summarized state); 9 also show the second mechanism and 1 shows neither. The second mechanism is checkable on all 52 shielded points: in **22/52** the *live downstream harness* re-triggers summarization anyway, equalizing the branches. Section 4.4 makes this division precise: a coalescence lemma and a one-line corollary guarantee that whenever the two branches coalesce, the resulting equality is exact — and we verify the occurrence directly in the recorded traces (the arguments explain 72/75 screened points; the 3 residuals, including 2 reward collisions, are declared). We also found genuine non-saturated synergy where the shield breaks (**1_vending_machine**: $C_H = C_M = -0.09$, $C_{HM} = 0$, so $I = +0.18$: either flip alone hurts, together they cancel).

Consequence for credit assignment. At every shielded point, single-layer harness credit $C(H)$ bills the harness for an effect the model absorbs: interaction-corrected credit $C_{HM} - C_M$ is zero while $C(H)$ is not. In the slack regime this is not an edge case — it is *all* measured points. Section 4.4 shows the error is not approximate: at screened points, $C(H) = -I(H, M)$ exactly.

4.4 Screening-off, formally

The measurements above are point identities, not expectations: under the premises of §4.1 (greedy decoding, fixed config, sequential serving, prefix caching off — with the resulting exactness *measured*, floor 0.0), the model, the environment, and the harness are all deterministic, so each replay branch is a single deterministic rollout. Fix a recorded trajectory with reward R_0 and a decision pair at index t : harness action h with alternative h' , model action a with sampled alternative a' . Writing $R_H = R(\text{do}(h'))$, $R_M = R(\text{do}(a'))$, $R_{HM} = R(\text{do}(h', a'))$ for the replayed rewards, the estimands are $C_H = R_0 - R_H$, $C_M = R_0 - R_M$, $C_{HM} = R_0 - R_{HM}$, and $I = C_{HM} - C_H - C_M$; throughout this section R is the test-fraction reward (not R_{eff} , whose token term accumulates over the differing prefixes), matching all measurements of §4.3. *Screening-off at the point* means $C_{HM} = C_M$.

Proposition 1 (Double-counting identity). *At any point, $C_{HM} - C_M = C_H + I$. Consequently, where screening-off holds: (i) the marginal credit of the harness flip in the presence of the model flip is zero, $C_{HM} - C_M = 0$; (ii) the single-layer harness credit equals the negated interaction, $C_H = -I$; (iii) a trainer that bills the harness C_H at such a point therefore assigns a signal consisting entirely of interaction — credit for an effect the model absorbs — while the interaction-corrected signal $C_{HM} - C_M$ vanishes.*

Proof. The first identity is the definition of I rearranged. Under $C_{HM} = C_M$, it gives $0 = C_H + I$; (i) and (iii) follow by definition. $\square$

Proposition 1 upgrades the empirical observation to a correction rule: the quantity a cross-layer trainer should bill the harness is $C_{HM} - C_M$ (one extra replay), which is exact in both regimes — equal to C_H where interaction is absent, and correctly zero where the model shields. Our training arm 3 uses precisely this signal.

Lemma 1 (Coalescence). *Let $s_k = (c_k, e_k)$ denote the agent state at step $k \geq t$ — model-visible context and environment (sandbox) state — and consider the two branches $B_M = \text{do}(a')$ and $B_{HM} = \text{do}(h', a')$. If the agent is deterministic and there exists $k \geq t$ with $s_k^{B_M} = s_k^{B_{HM}}$, then the branches coincide from k onward; in particular $R_M = R_{HM}$ and screening-off holds exactly.*

Proof. The one-step transition (model call, tool execution, harness policy) is a deterministic function of s_k . Equal states yield equal actions and equal successor states; induction to termination yields identical suffixes, hence identical final rewards. $\square$

Corollary 1 (Behavioral coalescence at the intervened turn). *If both branches execute identical action sequences from turn t to termination, then $R_M = R_{HM}$. (Harness context operations do not modify the environment, so the environment states at turn t agree; identical action suffixes applied to equal environment states under a deterministic environment yield equal final environment states, and R is a function of the final environment state.)*

The two empirical mechanisms of the anatomy are the two *coalescence-inducing events* we observe in practice; the lemma’s role is to guarantee that *whenever* coalescence occurs, the resulting equality is exact rather than approximate. Because reward equality does *not* certify coalescence (with reward resolution $1/n_{\text{tests}}$, collisions are possible), we verified the hypotheses directly in the recorded branch traces of all 75 screened points, comparing per-turn states (c_k, e_k) and per-turn behavior (action, observation), normalizing non-semantic identifiers that leak into test output (memory addresses, sandbox paths). The result, stated so that each count matches an enunciated argument: in **29/75** points the full state (c_k, e_k) becomes byte-identical within 1–4 turns of the intervention — Lemma 1’s hypothesis, verified; in **72/75** (a superset of the 29) the two branches execute identical action suffixes from the intervened turn itself — Corollary 1 applies and entails $R_M = R_{HM}$. The formal arguments therefore *explain 72 of the 75* measured screening events. The **3 residuals**: two (`l_log_parser` idx 10, in two configurations) are exactly the reward-collision case — $R_M = R_{HM}$ holds without trace coalescence, so the screening statistic counts them but the theory does not explain them — and one (`l_payroll_pipeline` idx 7 under g900, saturated at $R = 1.0$) coalesces behaviorally only 3 turns after the intervention, which is consistent with screening but not covered by Corollary 1; we declare all three rather than absorb them into the explained count. No suffix violated determinism: whenever states matched at some turn, the branches were identical from that turn to termination, corroborating the premises at trace level. *Re-triggering* (22/52): both branches executed the same a' , so environment states already agree; the live harness in B_M summarizes at step $t+1$, and context coalescence follows. *Re-injection* (23/24 annotated): a' (typically a `write_file` carrying the destroyed constants) equalizes the environment component, and the residual context difference is confined to content the greedy continuation never conditions on. The lemma thus explains a striking feature of the data: measured screening-off consists of *exact zeros*, not small values — determinism converts coalescence into equality, the measured 0.0 floor makes equality observable, and the coalescence itself is now measured rather than presumed.

Remark 1 (Pivotality and the competence window). Call a decision point *reward-pivotal* if at least one of the three interventions changes the reward. Non-pivotal points satisfy $C_H = C_M = C_{HM} = I = 0$ identically: nothing is detectable there, by construction rather than by noise. When the agent fails a task outright and no intervention — single or joint — rescues it, every point on that trajectory is non-pivotal. This is the formal content of the competence window: reward saturation does not merely add variance — it removes pivotal points,

which is why the Qwen3-1.7B pressure regime (11/12 saturated) had essentially no power and why the mt4 break rate collapses.

Corollary 2 (What single-layer training optimizes). *In a regime where screening-off holds at all measured points (our slack regime, 57/57), the per-decision signal of a single-layer harness trainer is $-I$ at every shielded point: it consists entirely of credit for effects the model absorbs. More generally, by Proposition 1 the single-layer signal errs by exactly I at every point where $I \neq 0$, regardless of regime — at the broken point `l_vending_machine`, $C_H = -0.09$ while the corrected signal is $+0.09$, an inverted sign. The corrected signal $C_{HM} - C_M$ is exact in both regimes.*

We emphasize the epistemic status of each piece. Proposition 1 is elementary algebra — its value is not mathematical depth but the correction rule it licenses. Lemma 1 holds given the determinism premises, and both the premises (§4.1) and the lemma’s hypothesis (this section) are *measured* rather than assumed. Whether coalescence occurs at a given point, which mechanism induces it, and how often screening-off holds at all are empirical questions that the rest of this section answered.

4.5 Control tasks and the `c_temp_label` exception

On control tasks, where summaries destroy nothing by construction, the method returns $C(H) = 0$ in 11/12 measured points — the construct behaves. The exception is informative: `c_temp_label` showed $C = +0.22$ at turn 0 despite all task-critical information surviving the summary. Audit of the replay pair shows the effect is *behavioral*, not informational: the summarized context changes the model’s subsequent formatting behavior, which a strict test then penalizes. $C(H)$ measures total causal effect through *any* pathway; the controls certify the informational pathway is clean, and the exception demonstrates the behavioral pathway exists and is detectable.

5 The Structure of Harness Credit

Can $C(H)$ be *predicted* from cheap features, so that replay is needed only to calibrate? We trained linear and GBM critics on 248 ground-truth points (features: turn, context size, position, test progress; task-clustered evaluation) against the baselines mandated by [8]: position and context-size heuristics. The honest answer is that **the critic never beats the heuristics** (Figure 4): on the combined L+V2 strata ($n = 161$, task-clustered Spearman) the GBM reaches 0.667 and the linear critic 0.675, against 0.708 for the position heuristic and 0.770 for `|context|`; pooled over all 248 points the linear critic’s 0.718 still trails the context-size heuristic. Per stratum, L is a tie (GBM 0.747 vs. position 0.741), in V2 the heuristic wins (0.693 vs. 0.796), and S saturates for everything (AUROC 1.0). For detection ($|C| > 0$) the GBM has a modest edge (AUROC 0.846 vs.

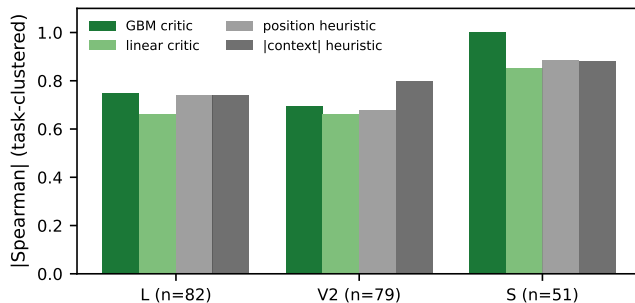


Figure 4: **F4 — Critics vs. trivial heuristics per stratum** (task-clustered $|\text{Spearman}|$). The critic never beats the best heuristic; $C(H)$ is structured by position and context size.

0.735 for position and 0.785 for the sign-inverted context-size heuristic) with overlapping CIs.

We report this not as a failed component but as a finding about the signal: **harness credit is mostly a function of context size and position**. That is exactly what the information-destruction mechanism predicts — how much is destroyed, and how late — and it means practitioners can get most of the ranking value of $C(H)$ from features they already log, reserving replay for calibration and for the interaction term, which no cheap feature predicts. The cross-environment test sharpens this: a critic trained on all $C(M)$ ground truth from the designed tasks and evaluated on the 94 MBPP+ $C(M)$ points (the only quantity with data on both sides; $C(H)$ has none in the MBPP+ runs) transfers at chance (Spearman 0.03 linear / -0.08 GBM, AUROC 0.30/0.50, MAE worse than a constant predictor; pre-registered secondary of D3). Amortized credit does not cross environments here — consistent with the replay-ground-truth bar of [8], and a further reason the training signal should be anchored in replay rather than in a learned proxy.

6 Training with Cross-Layer Credit

Does better credit train a better harness? We train the harness `context_policy` (a logistic policy over 5 features: bias, turn fraction, context tokens, test progress, message count) with four arms, **dose-matched in total rollouts** (episodes + credit replays): (1) *outcome-only* policy gradient; (2) *single-layer* $C(H)$ credit — an explicit reproduction of the CHILL-Harness principle in our environment; (3) *interaction-corrected* credit $C_{HM} - C_M$; (4) a *zero-credit* masking control (credit $\equiv 0$, no replays — which under our greedy tie-break at $\theta = 0$ is de facto an untrained keep-always policy; we state this explicitly). The pre-registered success criterion (GATE 3) required arm 3 to beat both arm 2 and arm 4. Calibration established the target is learnable in principle: on held-out tasks a

simple context-size threshold policy (thr600) achieves $R_{\text{eff}} = 0.398$, dominating keep-always (0.237) and summarize-always (−0.464); thr600 is expressible by the policy class.

Act 1: seed-dependent collapse. With raw features and $\text{lr} = 0.5$, every trained arm collapsed into one of the two fixed policies, with the attractor determined by seed (outcome: k/k/s; $C(H)$: s/s/k; $C_{HM} - C_M$: k/s/s). GATE 3 failed and we report it. Two hypotheses remained: the credit estimator is broken, or the optimization is. A pre-registered audit recomputed 60 logged credits by fresh two-greedy-replay: **58/60 sign agreement, zero sign flips toward harmful** (mean discrepancy −0.0015). The estimator is exonerated.

Act 2: stable optimization, no discovery. With the pre-registered fix (feature centering, $\text{lr} = 0.1$, gradient clipping), training is sane: gradients bounded, learned weights point the right way (negative bias, positive weight on context tokens, implicit decision thresholds at 2.5k–5.8k tokens). Yet **no arm in any seed discovers the threshold policy**: all non-collapsed runs converge to keep-always (held-out $R_{\text{eff}} = 0.237$ vs. thr600’s 0.398; Figure 5). Worse, credit-driven arms retain collapse risk the outcome-only arm avoids ($C(H)$: 1/3 seeds; $C_{HM} - C_M$: 1/3; outcome-only and zero: 0/3).

Diagnosis. Dose-matching in total rollouts means credit arms pay for replays with episodes: across 3 seeds the outcome arm saw 1,789 training episodes, $C(H)$ 786, $C_{HM} - C_M$ 319 (zero: 1,910). One might suspect the credit arms simply saw too little data. The state-visitation and reward-margin records rule this reading out and localize the failure in three steps. *First*, the learned policies are behaviorally inert: their implicit decision thresholds sit at 2,500–5,800 context tokens, but no decision in any arm or seed — 0 of 13,498, including the zero arm, de facto keep-always, the policy that maximizes context growth (1,910 episodes, cap 1,532 tokens) — and no held-out decision (cap 1,200 tokens) ever reaches that region: the learned thresholds can never fire, so every non-collapsed policy is keep-always in behavior. *Second*, in the states the training tasks *do* visit (thr600 fires above 600 tokens, well inside the visited range), the reward margin between thr600 and keep-always is statistically zero: paired bootstrap on the training tasks gives +0.024, CI95 [−0.026, +0.079], $P(\text{diff} \leq 0) = 0.19$ — there is no gradient signal to find, however accurate the credit. *Third*, thr600’s held-out dominance comes from cumulative-cost structure: long held-out tasks accumulate prompt tokens under keep-always (e.g. `1_vending_machine`, 5,933 cumulative prompt tokens, $R_{\text{eff}} = -1.12$) — a cost regime the training distribution never makes consequential. The train/held-out split was alphabetical, fixed before training and before we knew this gap existed. **The binding constraint is the training task distribution, not credit quality.** This sharpens the design lesson: a cross-layer training benchmark must verify that its training tasks assign a nonzero reward margin to the states where candidate policies actually differ — otherwise credit fidelity is moot, and the failure will masquerade

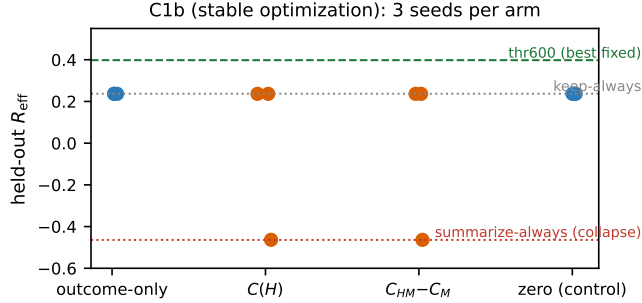


Figure 5: **F5 — Training outcome (C1b, stable optimization).** Held-out R_{eff} , 3 seeds per arm. No arm reaches the expressible thr600 policy; credit-driven arms (orange) carry collapse risk that outcome-only avoids.

as an algorithmic one.

7 Threats to Validity and Discussion

Synthetic tasks and external validity. Our tasks are designed, not sampled from a public benchmark, and this is a real limitation — but a motivated one. The measurement requires (i) control over the *position* of destroyable information, to separate informational from behavioral effects; (ii) per-stratum calibrated recoverability, which underlies the control tasks; and (iii) deterministic fractional reward, which underlies the exact floor. No public benchmark provides these jointly. We therefore ran the missing experiment in reduced form (pre-registered as D3 before running): a multi-turn MBPP+ adaptation — 60 tasks derived mechanically from MBPP+ [4] with fractional reward (6–10 test functions per task, canonical solutions validated in our sandbox), one slack and one pressure configuration on Qwen3-4B. The floor replicates exactly (1,294 null replays, all exact) and **screening-off replicates at 66/66 interaction points** — including a non-saturated point with $I = -0.4$, a live double-counting case in an external benchmark. What the adaptation cannot yet test is the pressure-regime break rate: 60/66 points are reward-saturated (Qwen3-4B is too strong for MBPP+), leaving 6 non-saturated points and no breaks — inconclusive at that power, the same saturation boundary seen at 1.7B and 8B. The designed tasks remain necessary for the regime-dependence claim; the core phenomenon — exact floor, exact screening, double counting — is not an artifact of our task design. We position this as a methods paper: the transferable artifact is the recorder/replay/forced-queue infrastructure, the APC finding, and the measurement protocol.

What replicates, raw, per model and environment. The floor replicates everywhere (pre-registered as D2c, D2d, D3 before running): the 8B con-

tributes 810 dedicated and 234 attached null replays and MBPP+ contributes 1,294, all exact. Screening-off replicates *unconditionally* on Qwen3-1.7B (0 raw breaks in 23 points) and on MBPP+ (66/66). On Qwen3-8B it does not: raw break rates are flat across regimes (5/28 slack, 6/34 mt6, 7/35 mt4), so the slack/pressure contrast established on Qwen3-4B does *not* transfer to the 8B, and we report this as the headline of the replication rather than conditioning it away. Saturation-based exclusion would be invalid here: clipping R at $\{0, 1\}$ can only *shrink* $|C_{HM} - C_M|$, so saturation can mask nulls but cannot fabricate breaks — a saturated break of 0.9 is a real break. The 8B slack breaks have a sharp common structure ($C_H = 1.0$, $C_M = 0.0$, $C_{HM} \in [0.8, 0.9]$, hence $I \in [-0.2, -0.1]$): the harness flip alone is catastrophic, the model flip alone is null, and the joint flip tracks the harness — the shield *inverts*. The mechanism is the one the anatomy of §4.3 identified: on the same tasks the 4B’s sampled a' re-injects all destroyed critical constants (5/5, 5/5, 6/6 on the shielded points), while the 8B’s a' re-injects none (0/5, 0/6, 0/8, 0/5; matching by constant-literal substring in the serialized a') — its solutions are structured so that the alternative action no longer carries the constants. The mechanism is verifiable on 4 of the 5 slack breaks (the fifth, `invoice_pricing`, has no annotated constants), and the breaks *recur*: all 5 slack break tasks break again, with identical per-task structure, in both pressure configurations (6/34 at mt6, 7/35 at mt4). Whether screening-off holds is therefore itself competence-dependent: the shield is a property of the model–harness pair (the closest precedent, model-dependent counterfactual *measurability* [8], concerns whether ground truth can be estimated at all; here the recovered causal structure itself inverts). We say competence, not scale: 4B and 8B differ in post-training as well as size, and the claim is anchored in the measured re-injection contrast on the original pool (which, as reported below, does not generalize as a mechanism), not in parameter count. Under pressure the 8B also shows a genuine non-saturated synergy break at `l_door_controller` ($C_{HM} = 0.23$ vs. $C_M = -0.46$, $I = +0.38$). Saturation still dominates both auxiliary models in opposite directions (11/12 of the 1.7B’s pressure points, from weakness; 90/97 of the 8B’s points, from strength — and the slack regime leaves the 8B a single non-saturated point, so the conditioned slack contrast has essentially no power and we do not claim it). The remaining step — the break rate on a pool curated to the 8B’s competence — was then run (pre-registrations 17 and 18): a first hardened pool failed its pre-registered curation rule (17/24 tasks solved at 1.0; reported as failure), and a second pool targeted at the fractional-reward profile passed (22 of 44 tasks in the (0.05, 0.95) window; 792/792 nulls exact after the ASLR fix of §4.1). **In-window, the 8B’s raw screening breaks replicate and stay flat across regimes** — 3/27 slack, 3/31 pressure — including a *non-saturated slack break* (`x_hours_bank`: $C_H = +0.25$, $C_M = -0.17$, $C_{HM} = 0.00$, $I = -0.08$), which closes the clipping objection: breaks persist where saturation cannot manufacture them. The pool-matched competence contrast is D2d’s (same 30 tasks: 4B 0/57 raw vs. 8B 5/28); within the curated pool the regime contrast seen on the 4B does not exist on the 8B even inside its competence window. Two of the three break points recur across both regimes with identical structure. Two

pre-registered sub-hypotheses failed and we report them as such: the binary re-injection mechanism does not generalize (at active slack points the curated-pool a' *never* re-injects constants, 0/15, yet only 3/15 break; under pressure 2 of 3 breaks occur *with* re-injection; one-sided Fisher $p = 1.0$ and $p = 0.94$ — the slack table is degenerate, no exposed points, so only the pressure test is informative, and it points the wrong way), and the directional prediction (pressure $\geq$ slack) failed (9.7% vs. 11.1%). All curated-pool break counts are descriptive: with 3 break tasks per regime we make no clustered inferential claim, and we make no cross-pool p -value comparison against the 4B’s 0/57. Two design threats remain and we name them: the curated comparison was, at the time of this analysis, cross-pool (the 4B’s 0/57 comes from the original pool — the paired contrast on identical tasks is D2d’s 0/57 vs. 5/28); and the curation window is operationalized by task *family* (the second pool was designed around the profile that survived curation), so in-window break rate could be a property of the family rather than of competence — the symmetric control (the 4B run on the same curated pool, pre-registered as D5) addresses both. The break structure is also heterogeneous: the D2d slack signature ($C_H = 1.0$, $C_M = 0.0$, $C_{HM} \approx 0.9$) does not recur — `h_cargo_manifest` tracks the harness ($C_{HM} = 0.46$ vs. $C_H = 0.62$) while `h_hotel_folio` tracks the model ($C_{HM} = 0.08$ vs. $C_M = 0.00$). A descriptive trace anatomy of all 6 curated-pool breaks (0 new rollouts) finds one uniform base and two patterns on top of it. Uniformly: every break is a `keep`→`summarize` flip at a `write_file` turn, the summary destroys essentially all critical constants in context (8/8, 5/6, 7/7, 10/10), and the harness flip alone is destructive. On top: either the sampled a' carries some constants and partially rescues the destroyed context (joint lands *between* the single flips: `h_hotel_folio`, `h_cargo_manifest`), or a' is itself beneficial and the destruction annuls exactly that benefit (joint returns to the original reward: `x_hours_bank`, `h_sku_validator` — the one positive I is of this kind). The two recurring points are numerically identical across regimes — same four arm rewards, same destroyed constants, same free-divergence off-sets, from different baseline trajectories — so the break is a property of the decision point (task $\times$ turn), not of the budget regime. The robust claim is therefore narrower than the mechanism story: *whether* screening-off holds is a property of the model-harness pair, but re-injection of destroyed information, which fully explained the annotated 4B/8B contrast of D2d, is neither necessary nor sufficient for the shield in a curated pool.

Clustered inference. Points within a task are not independent. At the task level the pressure-vs-slack contrast is 3 tasks vs. 0 (sign test $p = 0.125$); at the point level $p = 0.005$; the saturation-conditioned contrast behaves the same way (point-level $p = 0.037$; task-clustered permutation $p = 0.18$ – 0.25). We report all levels and lean on the screening-off sign test (75/78, $P = 2.6 \times 10^{-19}$) and cross-configuration recurrence of break points as the stronger evidence.

Estimator coupling. $C(M)$ and I share the sampled alternative action a' : they are not independent evidence, and the structural component of screening-off (§4.3) is partly a property of the estimand — a' sampled from the unsummarized state can re-inject information. We quantified this (23/24 annotated shielded points; 28 unannotated points declared) rather than hiding it; an estimand with a' sampled from the summarized state would measure a different, also legitimate, quantity.

Saturation is endogenous. Reward resolution is set by test count, so saturation — our mediator — is partly a design artifact. The competence-window analysis conditions on it explicitly; the inverted-U at mt4 is only partially explained (conditioned rate $0.045 \rightarrow 0.143$; the ordering $\text{mt8} > \text{mt6} > \text{mt4}$ persists), so the window remains a partially supported hypothesis, discussed but not claimed as a contribution.

Serving premises. The exact floor holds under fixed config, sequential requests, and APC off. It is a measured property of this stack, not a theorem; any deviation (batching, caching, config drift) voids it and must be re-measured.

8 Conclusion

We built the first apparatus that measures, on the same trajectory, the causal credit of harness decisions, model decisions, and their interaction, against an exactly-zero measured noise floor. The measurement yields a crisp, regime-dependent picture: with budget slack the model screens off the harness completely (0/57), so single-layer harness credit double-counts; under budget pressure the shield breaks at recurring points (8/65), inside a competence window shaped by reward saturation — and the regime contrast itself is a property of the model: a stronger model breaks screening at a flat rate in both regimes, in raw data and in a competence-curated pool. Better credit did not, however, train a better harness: a two-stage, pre-registered negative — with the estimator exonerated by audit — locates the binding constraint in the training task distribution itself: no reward margin where policies differ, unreachable states where learned thresholds would fire. We offer the infrastructure, the regime-dependence finding, the APC determinism warning, and a pre-registration ledger of 18 predictions (including 4 failures and 4 partials) as the contributions, and the training negative as the design brief for the benchmark this field still needs.

A Pre-Registration Ledger

All confirmatory analyses were pre-registered in the project’s execution plan before the corresponding data were collected. Table 2 lists all 18 pre-registrations with their outcomes, including the four that failed and the four partial. Failed predictions are reported in the main text where they occur.

#	Pre-registered prediction / rule	Outcome	Status
1	Score three per conflict = score &R; over scale	Q&R to 7-2-2-2-2 conflict, 3 models, 2 circumstances	held
2	Phase-A yield / accepting rule	followed as written	held
3	Critic evaluation results (task-clustered Spearman)	initial result reported ac-c	held
4	Pro/Con major vote to critic	followed	held
5	Structured α = distinct outcome, reported separately	followed	held
6	Critic score thresholded to limit volume	followed	held
7	Critic direction never aggregated	followed	held
8	Iteration analysis outside confidentiality on	not used	held
9	GLUE 1: aim 3 most best score 2 and 4	not used	failed
10	Scoring-off mechanism stop use	75/75, $P < 2.6 \times 10^{-19}$	held
11	Structure shift: one experiment on six aggregated models	50/50, 0 failures R&P	failed
12	Downsample: restricted break rate to pressure	limited 1/2 ($p < 0.001$)	failed
13	Critic primary outcome: held-out R&P > 0.30, 2/3/3 needs	no one reached it yet	failed
14	DD replication: Run, check pressure on Qwen2.5-72B	Run-check replicates: pressure mechanism	partial
15	DD replication: Run, check pressure on Qwen2.5-72B (not used)	Run replicates: run increasing time and 15/50 check breaks, should increase; 1 super-contrasted pressure break	partial
16	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
17	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
18	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
19	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
20	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
21	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
22	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
23	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
24	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
25	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
26	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
27	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
28	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
29	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
30	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
31	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
32	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
33	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
34	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
35	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
36	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
37	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
38	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
39	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
40	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
41	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
42	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
43	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
44	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
45	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
46	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
47	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
48	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
49	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
50	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
51	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
52	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
53	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
54	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
55	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
56	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
57	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
58	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
59	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
60	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
61	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
62	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
63	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
64	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
65	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
66	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
67	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
68	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
69	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
70	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
71	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
72	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
73	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
74	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
75	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
76	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
77	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
78	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
79	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
80	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
81	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
82	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
83	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
84	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
85	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
86	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
87	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
88	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
89	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
90	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
91	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
92	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
93	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
94	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
95	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
96	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
97	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
98	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
99	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial
100	DD model evaluation: Run, scoring, pressure on 100/100 P (100 tasks), critic transfer A-crit	Run scoring, pressure: 100/100 pressure mechanism: increasing, limited and	partial

Table 2: Pre-registration ledger. Failures are results, not omissions: #9 and #12 constitute the two-stage training negative (§6); #13 led to the competence-window analysis (§4.3).

References

- [1] Yanjun Chen, Yirong Sun, Hanlin Wang, Jinghan Wang, Xinming Zhang, Xiaoyu Shen, Wenjie Li, and Wei Zhang. Exact is easier: Credit assignment for cooperative LLM agents. arXiv:2603.06859v2, 2026.
- [2] Zhengyu Chen, Teng Xiao, Huaisheng Zhu, Yige Yuan, Luan Zhang, and Jingang Wang. Co-Harness: Co-evolving harnesses and model weights for LLM agents. arXiv:2607.22688, 2026.
- [3] Jiarun Fu, Lizhong Ding, Sida Chen, Honglei Xin, Chunhui Zhang, Pengqi Li, Qiuning Wei, Ye Yuan, and Guoren Wang. CHILL-Harness: Counterfactual harness learning for efficient reasoning in long-horizon agents. arXiv:2607.25825, 2026.
- [4] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] Haochen Luo, Yi Huang, Sichun Luo, Fengyuan Liu, Lei Li, Zefa Hu, Junlan Feng, and Qi Liu. Harness-aware self-evolving: Co-evolving model weights, harness, and task solutions. arXiv:2607.03935, 2026.
- [6] Xufang Luo et al. Agent lightning: Train ANY AI agents with reinforcement learning. arXiv:2508.03680, 2025.
- [7] Jaaneet Shah. Causal agent replay: Counterfactual attribution for LLM-agent failures. arXiv:2606.08275, 2026.
- [8] Haiyue Zhang. Credit without ground truth: Auditing step-level credit assignment in LLM agents against executed replay. arXiv:2608.19760, 2026.
- [9] Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, et al. Harness-Compass: Guiding automatic harness evolution toward generalizable and effective agent harnesses. arXiv:2608.01918, 2026.